



MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING



MODULE 3 UNIT 1
Lesson Video 2 Transcript

Module 3 Unit 1 Lesson Video 2 Transcript

Autonomy as a dial

PAUL MCDONAGH-SMITH: We've described the scaling paradox. Agentic AI is powerful enough to matter and fragile enough to require careful design. Now let's get practical, because the question every leader faces isn't, "Should we trust the agent?" but rather, "How much autonomy is appropriate for this specific task, in this specific context, with these specific stakes?" And that set of questions imply something important. Autonomy isn't a switch you flip; it's a dial you calibrate.

The autonomy spectrum

MCDONAGH-SMITH: Let me walk you through a framework that I think helps to clarify this. Imagine a spectrum, where at one end you have a fully constrained agent, a system that can only select from a pre-verified set of known correct responses. It matches a user's request to the closest validated answer. It can't hallucinate because it's not generating anything new or novel. It's safe, but it's also very limited. It can only handle situations that have been anticipated in advance.

Now, at the other end of the spectrum, you have a fully autonomous agent, a system that can generate novel outputs, take actions in external systems, and execute multistep plans without human intervention. It's very powerful, but every output carries uncertainty.

Most real-world deployments need to sit somewhere in between these two points on the spectrum. And the critical design decision, the one that determines your risk profile, is where on the spectrum you position each agent for each task in each context. Perhaps think of it this way: In the same organization, you might have a customer-facing agent operating near the constrained end, only surfacing answers that have been pre-validated. And an internal research agent operating further toward the autonomous end, where the cost of an error is lower and the value of flexibility is higher. The spectrum isn't one position for the whole enterprise. It's a portfolio of positions calibrated by context.

Engineering trust: The safeguard architecture

MCDONAGH-SMITH: Once you've positioned an agent on this spectrum, the next question is, well, what safeguards hold it there? Because the danger in scaling isn't only that an agent is given too much autonomy on day one, it's that autonomy creeps outward over time without a corresponding increase in oversight. There are several layers of safeguard that should be designed into any agentic system at scale.

Let's name four. Firstly, hardcoded constraints. These are non-negotiable boundaries built into the system architecture itself. They are not prompts the agent might override. They are rules that execute independently of the model's reasoning. If an agent is authorized to approve a transaction up to a certain threshold, but anything above that threshold requires human approval, that gate should be implemented outside the model, in the application logic, not in the prompt. The model can't talk its way past a hardcoded constraint. That's the point.

Then second, output verification. Before an agent's output reaches a user or triggers a downstream action, it can be checked against formal rules, consistency criteria, or a second model acting as a reviewer. This is analogous to a two-person rule in financial controls. The agent proposes and a verification layer confirms. It adds latency, but in high-stakes context, the tradeoff is regularly worth making.

And thirdly, reversibility by design. Wherever possible, agent actions should be designed to be reversible. If an agent sends a communication, can it be recalled? If it initiates a transaction, is there a rollback window? Designing for reversibility doesn't prevent errors, but it limits their potential blast radius. In safety-critical engineering, this is often a foundational principle. Never make an irreversible decision faster than your ability to detect that it was, in fact, wrong.

And then fourthly, escalation architecture. Every agent and agentic system should have a clear, tested escalation path, a mechanism for recognizing that it's reached the boundary of its confidence or its authority and routing the decision to a human or another system. The design challenge here is calibration. Escalate too often and you've rebuilt a manual process with extra steps. Escalate too rarely and you have an autonomous system operating beyond its competence.

The comparison that matters

MCDONAGH-SMITH: When evaluating the risk of an agentic system, the comparison isn't agent versus perfection. The comparison is agent versus the current process. And the current process is almost always human driven. And humans make mistakes. Inconsistency, unpredictability, often without audit trails. An agentic system that achieves 85% accuracy on a task where humans achieve 82%, but does so with complete traceability, consistent escalation, and a full audit log, may represent a significant improvement in actual risk management, even if it's not perfect. The question isn't, "Does the agent make errors?" The question is, "Does this agent with these safeguards produce better outcomes than the process it replaces?"

Conclusion

MCDONAGH-SMITH: Designing for uncertainty is not about eliminating risk; it's about making risk visible, bounded, and manageable. The autonomy spectrum gives you a language for these decisions. The safeguard layers give you an architecture for implementing them. And the comparison that matters, agent with guardrails versus current state process, gives you a realistic framing for evaluating the tradeoffs.

In the next video, we'll step back to the strategic level. How the economics of enterprise software are shifting, what that means for your scaling decisions, and how to build the organizational readiness to sustain agentic AI at scale.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.